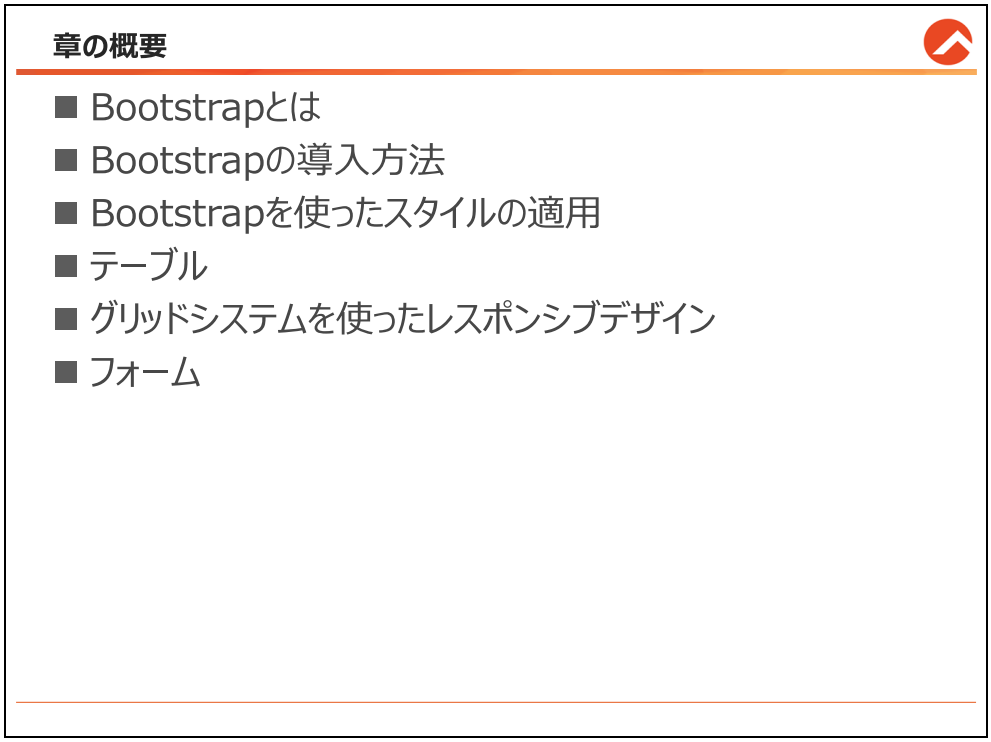
7

第7章 Bootstrap

章の目標

- Bootstrap の特徴と導入方法を習得する
- Bootstrap でよく使うスタイルについて実装できる
- Bootstrap でテーブルのカスタマイズができる
- グリッドシステムとレスポンシブデザインについて理解できる
- Bootstrap でフォームを実装できる

章の概要

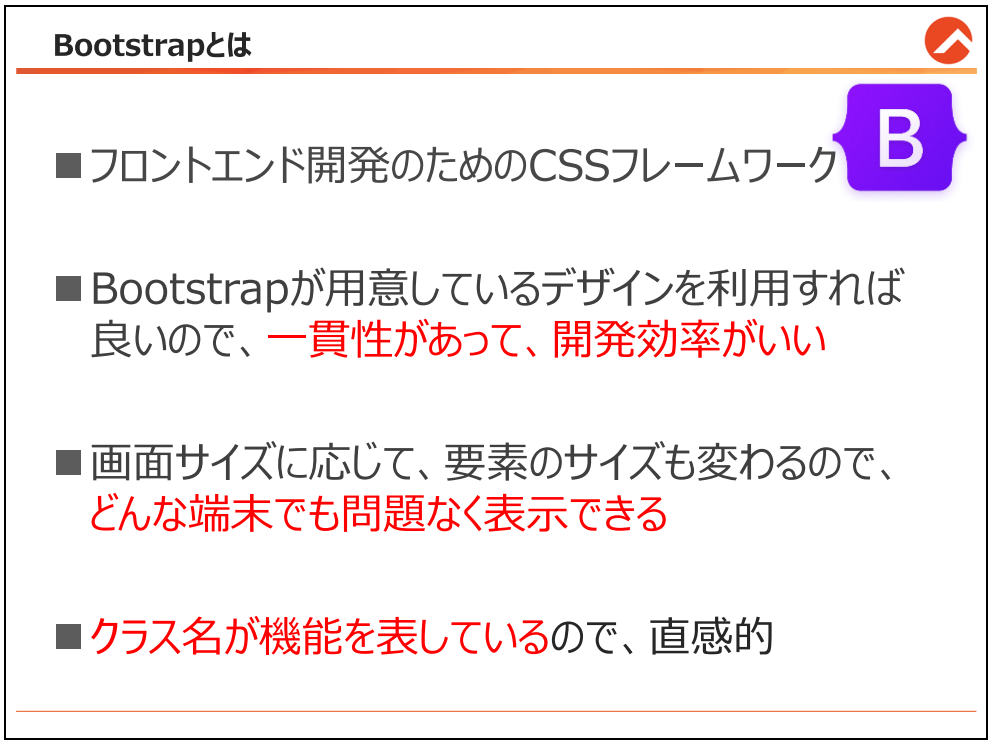


7 章では、Bootstrap の基本的な概要から導入方法、具体的な使い方までを学びます。

- **Bootstrap の概要**
  Bootstrap とは、統一されたデザインとレスポンシブなUI を簡単に作成するためのフロントエンドフレームワークです。
- **導入方法**
  CDN を利用した簡単な導入方法と、ローカルから読み込む方法を説明します。
- **具体的な使い方**
  グリッドシステム、ボタン、フォーム、テーブルなどの基本コンポーネントを用いた実践的な例を学びます。

この章を通じて、Bootstrap を効率よく活用するスキルを身に付けましょう。

7.1. Bootstrap とは



Bootstrap とは

Bootstrapは、WebサイトやWebアプリケーションのフロントエンド開発を簡単にするためのオープンソースのフレームワークです。もともと(旧)Twitter 社の社内プロジェクトとして始まり、現在は多くの開発者に利用されています。

特徴:

- モバイルファースト: 最初からスマートフォンでの表示を意識して設計されています。
- 統一されたデザイン: 一貫性のある UI を素早く構築可能です。
- レスポンシブデザイン: さまざまな画面サイズに適応します。

本書では、(2026 年 11 月 5 日基準)最新バージョンである 5.3.8 バージョンを利用します。

システム開発における外見の課題

システム開発では、機能の実装だけでなく、ユーザが使いやすい見た目やデザインも重要です。しかし、既存の方法で外見を整えるのは、次のような課題があります。

既存の方法: 手動でCSS 記述

たとえば、以下のように「青いテキスト」を表示したい場合、CSS を手動で記述する必要があります。

HTML

```HTML
<!-- CSS を直接記述 -->
<style>
    .blue-text {
        color: blue;
    }
</style>
<p class="blue-text">青いテキスト</p>
```

もしくは、インラインスタイルを使う方法もあります。

HTML

```HTML
<!-- インラインスタイル -->
<p style="color: blue;">青いテキスト</p>
```

この方法の課題

1. **時間がかかる:**テキストの色やサイズを変更するたびにCSS を記述する必要があります。
2. **保守性が低い:**クラス名やインラインスタイルが増えると、どこで定義されているか追跡するのが大変です。
3. **知識が必要:**CSS の構文を理解していないと適切な記述ができません。
4. **マルチデバイス対応が大変:**
   スマホやタブレット、PC で異なるレイアウトを実装する必要があり、工数が増えます。

Bootstrap での解決

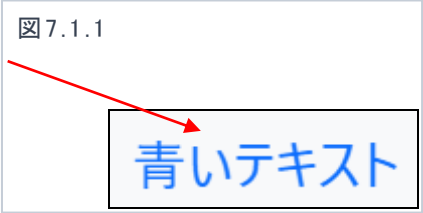
Bootstrap を使えば、CSS を記述する手間が省けます。HTML の**要素に特定のクラスを付与するだけ**で、スタイルが適用されます。

＜同じ「青いテキスト」をBootstrap で実装する場合＞

HTML

```
<p class="text-primary">青いテキスト</p>
```

図7.1.1



他の例（ボタンのデザイン）

Bootstrap を使えば、以下のようなデザインも簡単に実装できます。

従来の方法:

HTML

```
<style>
    .custom-button {
        background-color: #007bff;
        color: white; /* ほかに padding や border などを指定 */
    }
</style>
<button class="custom-button">送信</button>
```

Bootstrap を使う方法:

HTML

```
<button class="btn btn-primary">送信</button>
```

図7.1.2



Bootstrap を使うメリット

1. 短時間で実装可能:

クラスを付与するだけで、すぐにデザインが適用されます。

2. 一貫性のあるデザイン:

Bootstrap には統一されたスタイルが用意されているため、プロジェクト全体でデザインが統一されます。

3. CSS の記述量の減少:

CSS ファイルを記述する頻度が激減し、誰でも簡単にデザインを適用できます。

4. 直感的に使える:

クラス名で機能を表しているため、初心者でも簡単に理解できます。

5. レスポンシブデザインが簡単

スマホやタブレット、PC、どの端末でも最適な表示ができます。

7.2. Bootstrap の導入方法

Bootstapの導入方法

- CDN又はローカルから専用のCSS・JSファイルを読み込む
- CDNの方が簡単に導入できるが、インターネット環境必須
- ローカルの方は、仕様のカスタマイズが可能

```
<head>
<!--CDNからBootstrap CSS導入-->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet">

<!--ローカルからBootstrap CSS導入 -->
<link rel="stylesheet" th:href="@{/css/bootstrap.min.css}" />
</head>
```

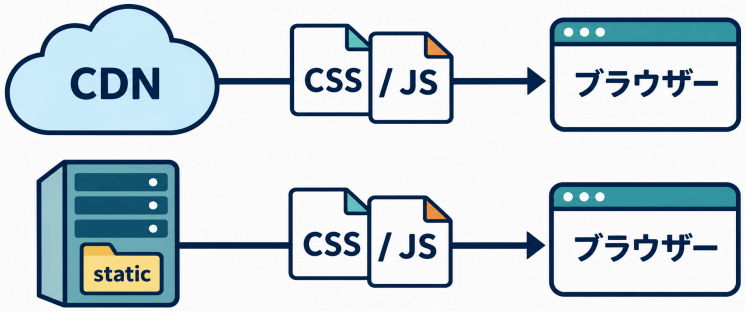
```
<!- CDNからBootstrapのJSを導入-->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js">
</script>

<!- ローカルからBootstrapのJSを導入-->
<script th:src="@{/js/bootstrap.bundle.min.js}"></script>
</body>
```

Bootstrap をプロジェクトに導入する方法は、次の2 つがあります：

1. CDN（Content Delivery Network）を利用する方法
2. ローカル（Spring プロジェクト内）に導入する方法

図7. 2. 1



BootstrapのCSS・JSは、CDNまたはアプリ内のstaticからブラウザーへ読み込みます。

※バージョンによって、詳細が変わる可能性がありますので、以下の公式ホームページのドキュメントを参照してください。

https://getbootstrap.jp/

CDN を利用した導入

CDN（Content Delivery Network）を利用すると、簡単にBootstrap を導入できます。CDNとは、世界中に配置されたサーバーのネットワークを利用して、コンテンツを配信する仕組みです。Bootstrap の公式CDN を使うことで、ローカルにファイルをダウンロードする必要がなく、すぐに使い始めることができます。手順は下記となります。

① <head>要素内に、<meta name="viewport">を記載

HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

モバイルデバイスでの適切なレスポンシブ動作のために記載する必要があります。

② Bootstrap のcss ファイルの読み込む

HTML

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet"
integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
```

<head>内に上記のURL を記載します。

③ Bootstrap のjsファイルを読み込む

HTML

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
crossorigin="anonymous">
</script>
```

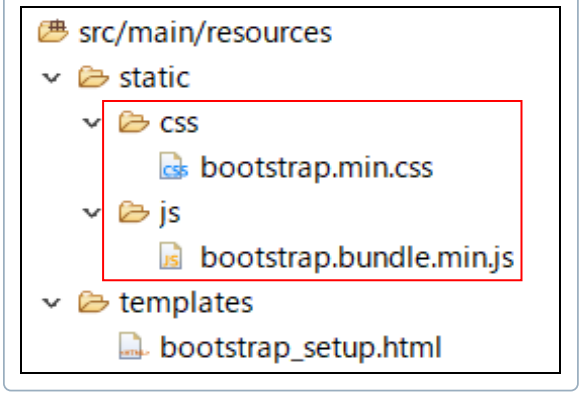
<body>の末尾に上記のURL を記載します。

Bootstrap をWeb サイト内で利用したいなら、上記のコードは必須です。**URL は、公式ホームページからお勧めのURL を参照することができます。**

ローカル(Spring プロジェクト内) からの導入

5 章で学習した通り、Thymeleaf 上でCSS・JS ファイルを読み込むことができます。事前にBootstrap のCSS・JS ファイルをダウンロードし、「static」フォルダ内に格納しておくと、格納場所を指定して、読み込むことができます。

図7. 2. 2



① 以下のコードを<head>に記入

HTML

```
<link rel="stylesheet" th:href="@{/css/bootstrap.min.css}" />
```

② 以下のコードを<body>の末尾に記入

HTML

```
<script th:src="@{/js/bootstrap.bundle.min.js}"></script>
```

下記は、Bootstrap を導入したシンプルなHTML の例です。

Java …/controller/ProductController.java

```
//省略

@GetMapping("/7-2-1")
        public String showBootStrapSetup() {
                return "bootstrap_setup";
        }
```

html ファイルに遷移させるために、遷移用メソッドを定義します。

HTML …/resources/templates/bootstrap_setup.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
<meta charset="UTF-8" />
<!-- ①以下のコードを記入することで、モバイル端末での表示を最適化できます。-->
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Bootstrap Example</title>

<!-- ② Bootstrap CSS をローカルから読み込む -->
<link rel="stylesheet" th:href="@{/css/bootstrap.min.css}" />

<!-- ② Bootstrap CSS をCDN から読み込む -->
<link href=
"https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
rel="stylesheet"
integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
</head>

<body>
<!-- コンテンツ -->
<div class="container">
    <h1>Hello, Bootstrap!</h1>
    <button class="btn btn-primary">ボタン</button>
</div>

<!-- ③ <body>の末尾に、Bootstrap のJS をローカルから読み込む-->
<script th:src="@{/js/bootstrap.bundle.min.js}"></script>

<!-- ③ <body>の末尾に、Bootstrap のJS をCDN から読み込む -->
```

HTML …/resources/templates/bootstrap_setup.html 続き

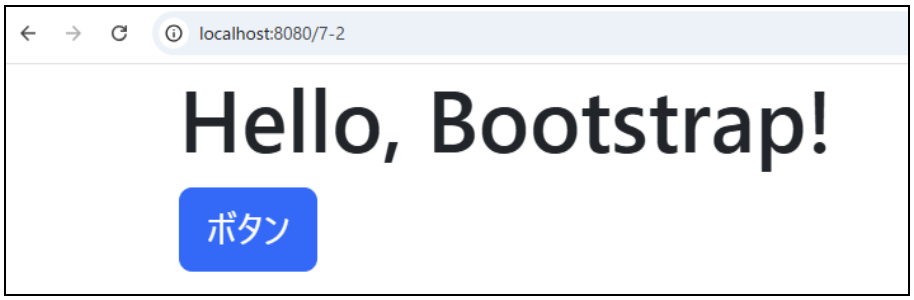
```
32  <script src=
33  "https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.
    bundle.min.js"
34  integrity="sha384-
    FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI
    "
35  crossorigin="anonymous">
36  </script>
37  </body>
38  </html>
```

サンプルコードには、両方の方法を記述していますが、実際は **CDN とローカルのうち、どちらかのみ記述**してください。**本研修では、ローカルからの読み込みを採用**します。

動作確認

http://localhost:8080/7-2

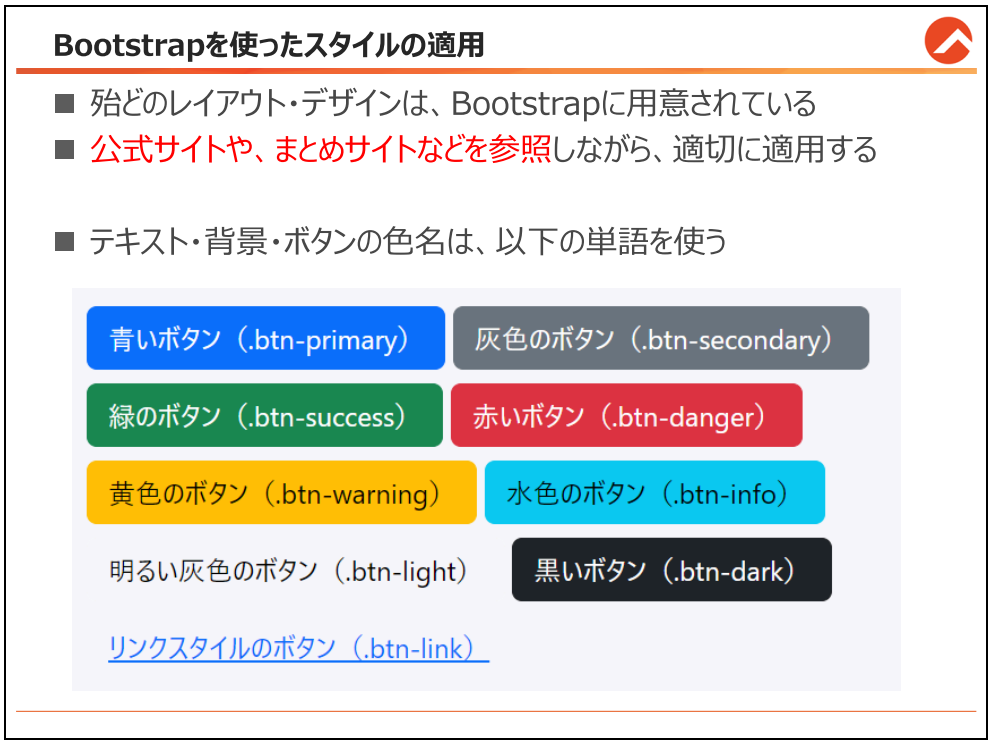
図7. 2. 3



青いボタンが表示されたら、問題なく導入ができているということです。

最新情報は公式サイト（https://getbootstrap.jp/）で確認してください。

7.3. Bootstrap を使ったスタイルの適用



Bootstrap は、Web サイトでよく使われるデザインや機能があらかじめ用意されています。特に、クラスをHTML に付与するだけで簡単にスタイルを適用できる点が大きな特徴です。

すべてのスタイルを一度に学ぶことは難しいですが、この節では特によく使うスタイルをいくつか紹介します。また、詳細な情報は以下の公式サイトや参考サイトで確認できます。

- Bootstrap 公式サイト(https://getbootstrap.jp/)
- とほほのBootstrap 5 入門(https://www.tohoho-web.com/bootstrap5/index.html)

7.3.1. テキスト

Bootstrap では、テキストの配置を簡単に変更できます。● **左揃え**: .text-start● **中央揃え**: .text-center● **右揃え**: .text-end**…/resources/templates/style_sample.html**

HTML

```
<h2 class="text-center">配置（Alignment）</h2>

    <p class="text-start">左揃え（.text-start）</p>
    <p class="text-center">中央揃え（.text-center）</p>
    <p class="text-end">右揃え（.text-end）</p>
```

図7. 3. 1



※CSS の text-align プロパティと同じく、ブロック内のインライン要素(テキスト、イメージ)の位置を調整できます。ブロック要素を動かすことはできません。

7.3.2. 色

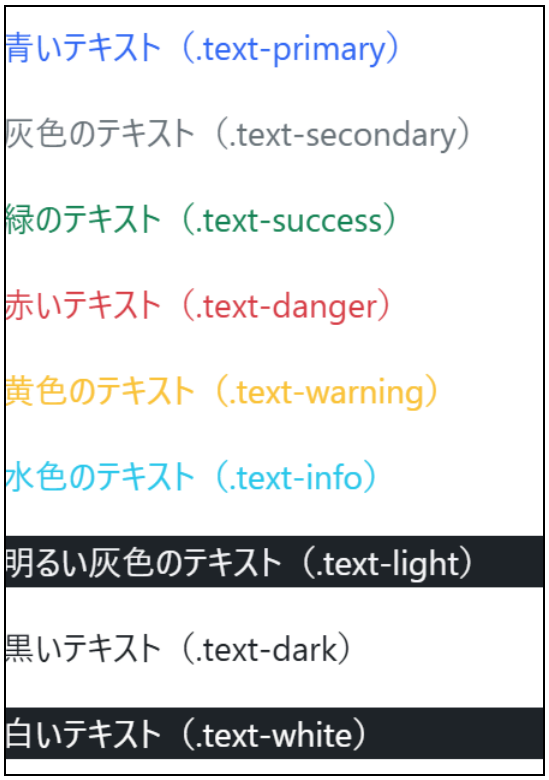
色のスタイルは、テキスト、背景、ボーダーなどに簡単に適用できます。まずは、色を変更する基本を学びましょう。

■ **テキストの色**Bootstrap の色クラスは「色名」ではなく、**「その色がよく使われる用途」に基づいています**。

HTML …/resources/templates/style_sample.html

```
<p class="text-primary">青いテキスト（.text-primary）</p>
    <p class="text-secondary">灰色のテキスト（.text-secondary）</p>
    <p class="text-success">緑のテキスト（.text-success）</p>
    <p class="text-danger">赤いテキスト（.text-danger）</p>
    <p class="text-warning">黄色のテキスト（.text-warning）</p>
    <p class="text-info">水色のテキスト（.text-info）</p>
    <p class="text-light bg-dark">明るい灰色のテキスト（.text-light）</p>
    <p class="text-dark">黒いテキスト（.text-dark）</p>
    <p class="text-white bg-dark">白いテキスト（.text-white）</p>
```

図7. 3. 2



■ 背景色

背景色のクラスは、テキストの色クラスと同じ命名ルールです。**text-がbg-に変わるだけ**です。

薄い色は、「-subtle」を追記することで表現できます。（Bootstrap 5.3 以降の機能）

HTML …/resources/templates/style_sample.html

```
<div class="bg-primary text-white">青い背景（.bg-primary）</div>
<div class="bg-secondary text-white">灰色の背景（.bg-secondary）</div>
<div class="bg-success text-white">緑の背景（.bg-success）</div>
<div class="bg-danger text-white">赤い背景（.bg-danger）</div>
<div class="bg-warning text-dark">黄色の背景（.bg-warning）</div>
<div class="bg-info text-white">水色の背景（.bg-info）</div>
<div class="bg-light text-dark">明るい灰色の背景（.bg-light）</div>
<div class="bg-dark text-white">黒い背景（.bg-dark）</div>
<div class="bg-white text-dark">白い背景（.bg-white）</div>
<div class="bg-primary-subtle">薄い青い背景（.bg-primary-subtle）</div>
<div class="bg-secondary-subtle">薄い灰色（.bg-secondary-subtle）</div>
<div class="bg-success-subtle">薄い緑の背景（.bg-success-subtle）</div>
<div class="bg-danger-subtle">薄い赤い背景（.bg-danger-subtle）</div>
<div class="bg-warning-subtle">薄い黄色の背景（.bg-warning-subtle）</div>
<div class="bg-info-subtle">薄い水色の背景（.bg-info-subtle）</div>
<div class="bg-light-subtle">薄い明るい灰色（.bg-light-subtle）</div>
<div class="bg-dark-subtle">薄い黒い背景（.bg-dark-subtle）</div>
```

図7. 3. 3


青い背景（.bg-primary）
灰色の背景（.bg-secondary）
緑の背景（.bg-success）
赤い背景（.bg-danger）
黄色の背景（.bg-warning）
水色の背景（.bg-info）
明るい灰色の背景（.bg-light）
黒い背景（.bg-dark）
白い背景（.bg-white）
薄い青い背景（.bg-primary-subtle）
薄い灰色の背景（.bg-secondary-subtle）
薄い緑の背景（.bg-success-subtle）
薄い赤い背景（.bg-danger-subtle）
薄い黄色の背景（.bg-warning-subtle）
薄い水色の背景（.bg-info-subtle）
薄い明るい灰色の背景（.bg-light-subtle）
薄い黒い背景（.bg-dark-subtle）

7.3.3. ボタン

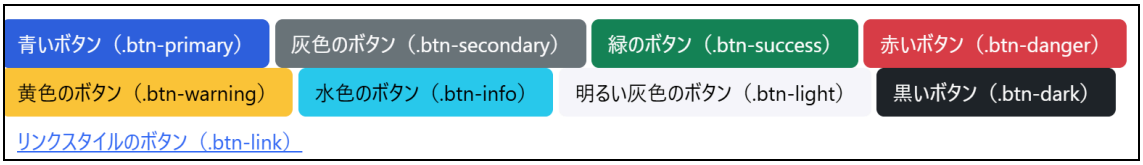
Bootstrap を使えば、HTML のボタン要素にクラスを追加するだけで、簡単にデザインを適用できます。ここでは、基本的なボタンの色やサイズ、状態について解説します。

■ **ボタンの色**Bootstrap では、**btn クラスを基礎として、さらに色を指定するクラスを追加**することで、簡単にさまざまな色のボタンを作成できます。

HTML …/resources/templates/style_sample.html

```
1  <button class="btn btn-primary">青いボタン（.btn-primary）
   </button>
2  <button class="btn btn-secondary">灰色のボタン（.btn-
   secondary）
3  </button>
4  <button class="btn btn-success">緑のボタン（.btn-success）
   </button>
5  <button class="btn btn-danger">赤いボタン（.btn-danger）
   </button>
6  <button class="btn btn-warning">黄色のボタン（.btn-warning）
   </button>
7  <button class="btn btn-info">水色のボタン（.btn-info）</button>
8  <button class="btn btn-light">明るい灰色のボタン（.btn-light）
9  </button>
10 <button class="btn btn-dark">黒いボタン（.btn-dark）</button>
11 <button class="btn btn-link">リンクスタイルのボタン（.btn-link）
12 </button>
```

図7. 3. 4



※btn-link は見た目がリンクのようになります。

※<button>だけでなく、<input>や<a>タグにも適用可能です。

■ **ボタンのサイズ**btn クラスに加えて、サイズを指定するクラスを追加することで、大きなボタンや小さなボタンを作成できます。

HTML …/resources/templates/style_sample.html

```
1  <button class="btn btn-dark btn-lg">大きなボタン（.btn-lg）
   </button>
2  <button class="btn btn-dark">普通サイズボタン</button>
3  <button class="btn btn-dark btn-sm">小さなボタン（.btn-sm）
   </button>
```

図7. 3. 5



■ **無効化されたボタン**Bootstrap では、ボタンを無効化（クリックできない状態に）するためのクラスが用意されています。条件によって、無効化・有効化を切り替えする実装の時使われます。

HTML …/resources/templates/style_sample.html

```
1  <button class="btn btn-secondary disabled">無効化されたボタン
2  </button>
```

図7. 3. 6



※マウスで当ててみると、クリックアニメーションが発生しないことがわかります。

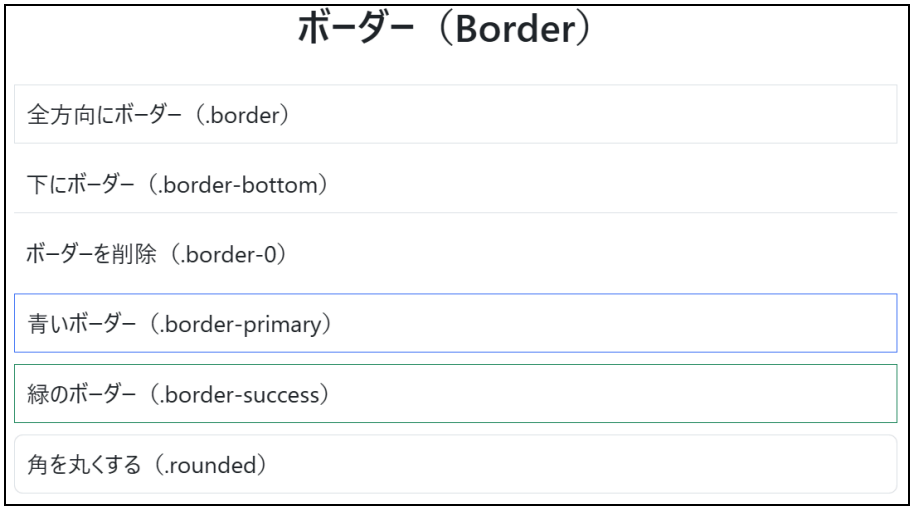
7.3.4. ボーダー(Border)

ボーダーは、要素に枠線を追加するためのクラスです。Bootstrap では、枠線の追加、削除、色の指定、角の丸みなどを簡単に設定できます。

HTML

```
<h2 class="text-center mb-4">ボーダー（Border）</h2>
<div class="border p-2 mb-2">全方向にボーダー（.border）</div>
<div class="border-bottom p-2 mb-2">下にボーダー（.border-bottom）
</div>
<div class="border-0 p-2 mb-2">ボーダーを削除（.border-0）</div>
<div class="border border-primary p-2 mb-2">青いボーダー
 （.border-primary）</div>
<div class="border border-success p-2 mb-2">緑のボーダー
 （.border-success）</div>
<div class="border rounded p-2 mb-2">角を丸くする（.rounded）</div>
```

図7. 3. 7



※p や mb のクラスは、次のページで説明します。

※border-0 は条件によって、枠線の有無を決める実装の時使われます。

7.3.5. 余白

■ **マージン（Margin）**マージンは、要素の**外側の余白**を調整するために使用します。

クラスの命名ルール● m-0 ～ m-5: 全方向のマージンを設定● mt-（上）、mb-（下）、ms-（左/スタート）、me-（右/エンド）で特定の方向に設定● mx-（左右）、my-（上下）でまとめて設定

HTML

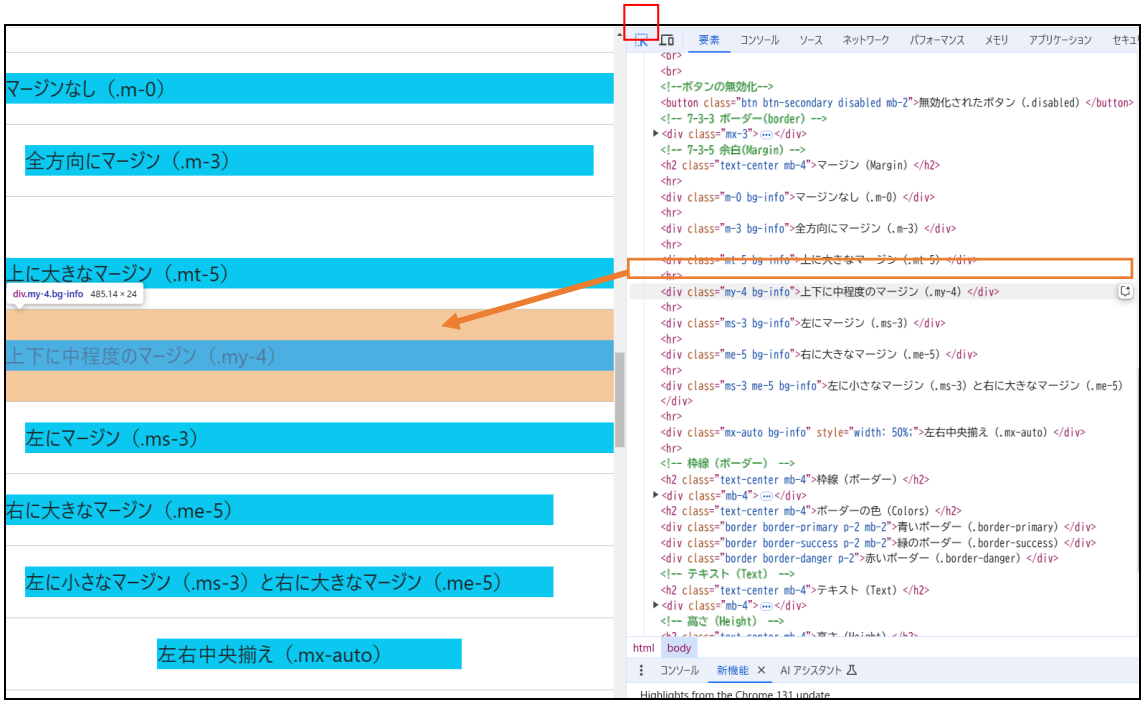
```
<hr>
<div class="m-0 bg-info">マージンなし（.m-0）</div>
<hr>
<div class="m-3 bg-info">全方向にマージン（.m-3）</div>
<hr>
<div class="mt-5 bg-info">上に大きなマージン（.mt-5）</div>
<hr>
<div class="my-4 bg-info">上下に中程度のマージン（.my-4）</div>
<hr>
<div class="ms-3 bg-info">左にマージン（.ms-3）</div>
<hr>
<div class="me-5 bg-info">右に大きなマージン（.me-5）</div>
<hr>
<div class="ms-3 me-5 bg-info">左に小さなマージン（.ms-3）と右に大きな
マージン（.me-5）</div>
<hr>
<div class="mx-auto bg-info" style="width: 50%;">左右中央揃え（.mx-auto）
</div>
<hr>
```

※<hr>は、水平線を引く要素です。余白の範囲の参考のため記述されています。（デフォルトで少しの上下の余白が生成されます）

要素の余白を簡単に調べられる方法についてご紹介します。

① 「F12」で「開発者ツール」を開きます。② 以下の赤枠のボタンをクリックします。（又は Ctrl+Shift+C）③ 「要素」タブの調べたい HTML コードにマウスを当てます。※ マウスを<div class = “my-4 bg-info”>上下に中程度のマージン(.my-4) に当てたので、余白の範囲が表示されました。

図7. 3. 8



■ パディング（Padding）

パディングは、**要素の内側の余白**を調整するために使います。Bootstrap では、p-を使ったクラス名を指定するだけで簡単に設定できます。

図7.3.9



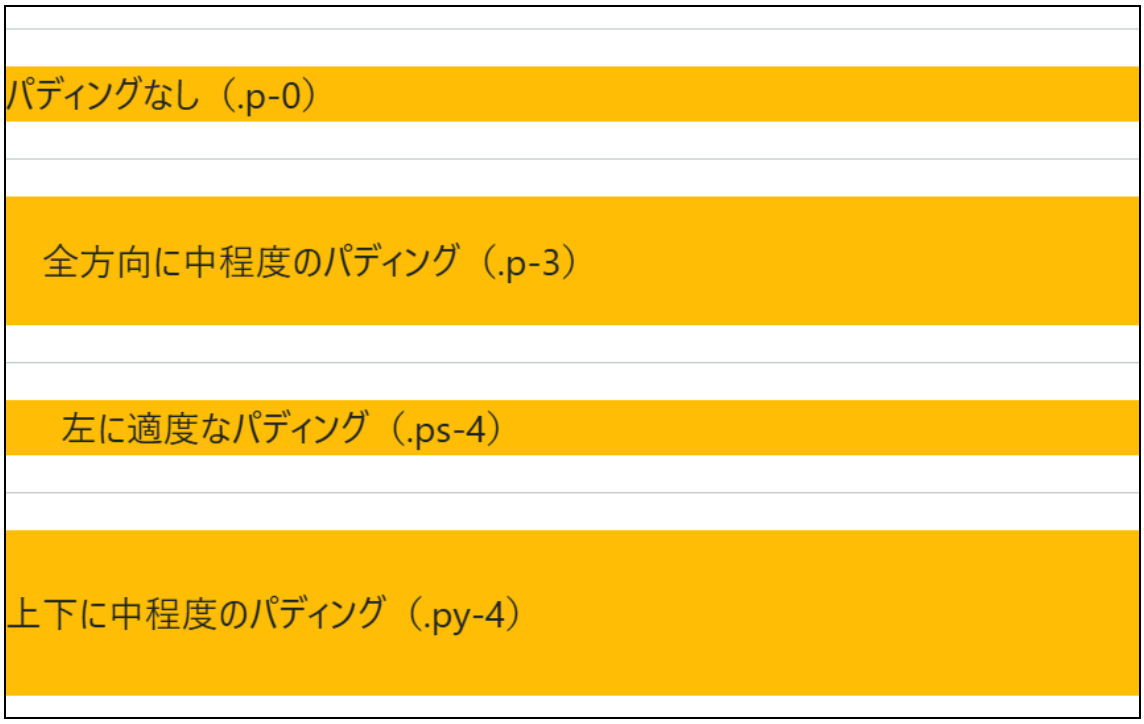
マージンは枠の外側、パディングは枠と内容の間の余白です。

- p-0 ～ p-5　全方向のパディングを設定
- pt-0 ～ pt-5 上のパディングを設定
- pb-0 ～ pb-5 下のパディングを設定
- ps-0 ～ ps-5 左のパディングを設定（start）
- pe-0 ～ pe-5 右のパディングを設定（end）
- px-0 ～ px-5 左右のパディングを設定
- py-0 ～ py-5 上下のパディングを設定

HTML

```
<hr>
<div class="bg-warning p-0">パディングなし（.p-0）</div>
<hr>
<div class="bg-warning p-3">全方向に中程度のパディング（.p-3）</div>
<hr>
<div class="bg-warning ps-4">左に適度なパディング（.ps-4）</div>
<hr>
<div class="bg-warning py-4">上下に中程度のパディング（.py-4）</div>
<hr>
```

図7. 3. 10


パディングなし（.p-0）
全方向に中程度のパディング（.p-3）
左に適度なパディング（.ps-4）
上下に中程度のパディング（.py-4）

7.3.6. ブロック要素の幅の調整(高さと横幅)

Web ページのデザインでは、ブロック要素の高さや横幅を適切に調整することが重要です。Bootstrap では、簡単にこれらを設定するクラスが用意されています。この節では、「高さ」と「横幅」の調整方法を説明します。

■ 高さ(Height)の調整

- h-25 親要素の高さの 25%を設定● h-50 親要素の高さの 50%を設定● h-75 親要素の高さの 75%を設定● h-100 親要素の高さの 100%を設定● h-auto コンテンツに応じた自動調整※h-35 のような微調整は別途カスタマイズが必要です。

ブロック要素の高さを設定する場合、**親要素の高さ**が重要です。子要素の高さは、親要素の高さに依存して計算されます。

HTML

```
<!-- 親要素 -->
<div class="bg-danger" style="height: 200px;">
        <hr>
        <!-- 子要素 -->
        <div class="bg-info h-25">高さ25%（子要素: .h-25）</div>
        <div class="bg-warning h-50">高さ50%（子要素: .h-50）</div>

</div>
```

高さ 200px 固定の親要素(赤いブロック)がある場合、子要素に h-25 と h-50 と指定すると、200px の 25%と 50%の高さが適用されます。

図7. 3. 11



■ 横幅(Width)の調整

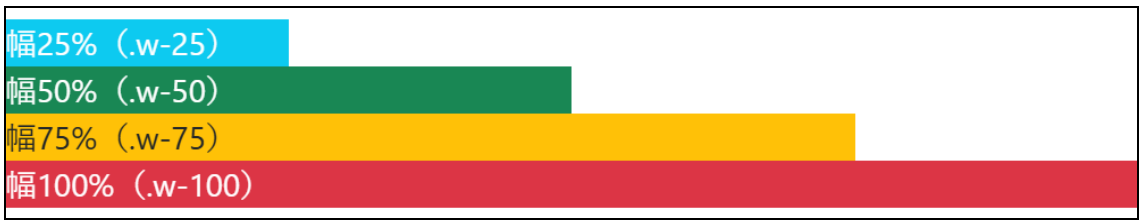
高さと同様に、ブロック要素の横幅もBootstrap 上で調整できます。

● w-25 親要素の幅の25%● w-50 親要素の幅の50%● w-75 親要素の幅の75%● w-100 親要素の幅の100%（全幅）● w-auto コンテンツに応じた自動幅

HTML

```
<div class="bg-info text-white w-25">幅25%（.w-25）</div>
<div class="bg-success text-white w-50">幅50%（.w-50）</div>
<div class="bg-warning text-dark w-75">幅75%（.w-75）</div>
<div class="bg-danger text-white w-100">幅100%（.w-100）</div>
```

図7. 3. 12



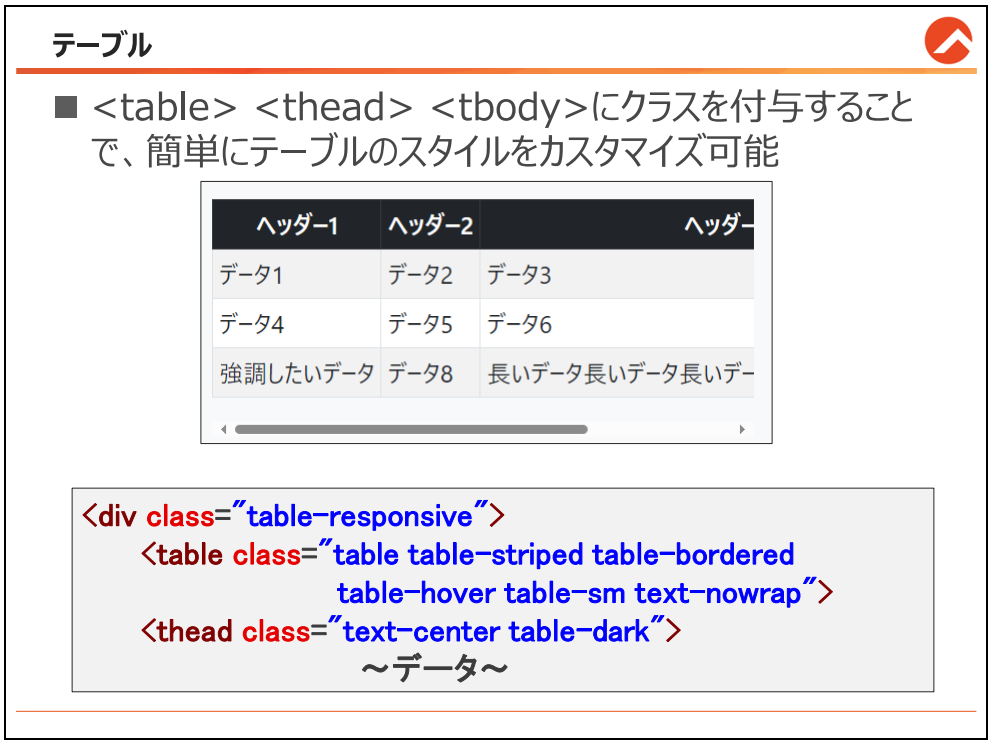
動作確認

http://localhost:8080/7-3

是非実行して、実際の動きを確認してください。

今回のサンプルは、あくまでよく使うクラスの一例ですので、紹介できていないクラスがたくさん残っています。他の節で、少しずつ新しいクラスを紹介していきますが、それでも全てを網羅することが難しいので、公式サイトや、まとめサイトで必要なクラスを調査して適用させるのが、Bootstrap を極める第一歩です。

7.4. テーブル



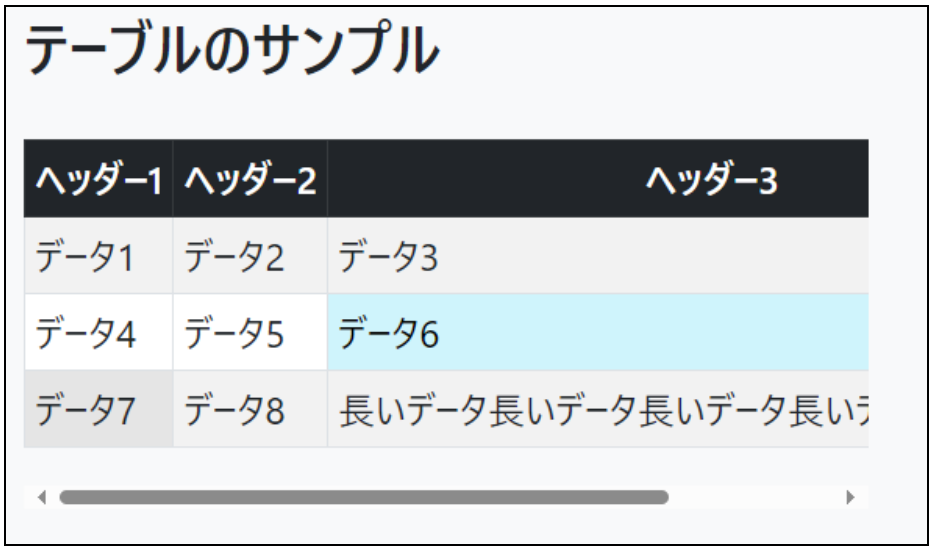
Bootstrap のテーブルクラス

Bootstrap を使用することで、テーブルのスタイルを簡単にカスタマイズすることができます。基本的なクラスは以下の通りです。

- **table**: 基本的なテーブルスタイルを適用します。● **table-striped**: 各行を交互に色付けして、視認性を向上させます。● **table-bordered**: テーブルに枠線を追加します。● **table-hover**: 行にカーソルを合わせたときに強調表示します。● **table-dark**: テーブル全体に暗いテーマを適用します。● **table-sm**: コンパクトなテーブルスタイルを適用します。● **table-responsive**: テーブルを狭い画面でもスクロール可能にします。table 要素を別途の div で囲う必要があります。● **table-active**: アクティブな行やセルを強調するためのクラス。● **table-primary(色名)**:状態に応じた行やセルの背景色を設定するためのクラス。**text-nowrap**: テキストの折り返しを防ぎ、一行で表示します。このクラスをテーブルのヘッダーやセルに適用することで、テキストが縦に表示されるのを防ぎます。

※その他に、テーブルの**ヘッダーとデータ行のスタイルを個別**で行うことができるように、通常は、<table>の中に**<thead>と<tbody>要素**を分けて実装します。

図7. 4. 1



HTML …/resources/templates/table_sample.html

```
 <h2 class="mb-4">テーブルのサンプル</h2>

<div class="table-responsive">
    <table class="table table-striped table-bordered table-hover table-sm
                    text-nowrap">
        <thead class="text-center table-dark">
            <tr>
                <th>ヘッダー1</th>
                <th>ヘッダー2</th>
                <th>ヘッダー3</th>
            </tr>
        </thead>
        <tbody>
            <tr class="table-info">
                <td>データ1</td>
                <td>データ2</td>
                <td>データ3</td>
            </tr>
            <tr >
                <td>データ4</td>
                <td>データ5</td>
                <td>データ6</td>
            </tr>
```

HTML

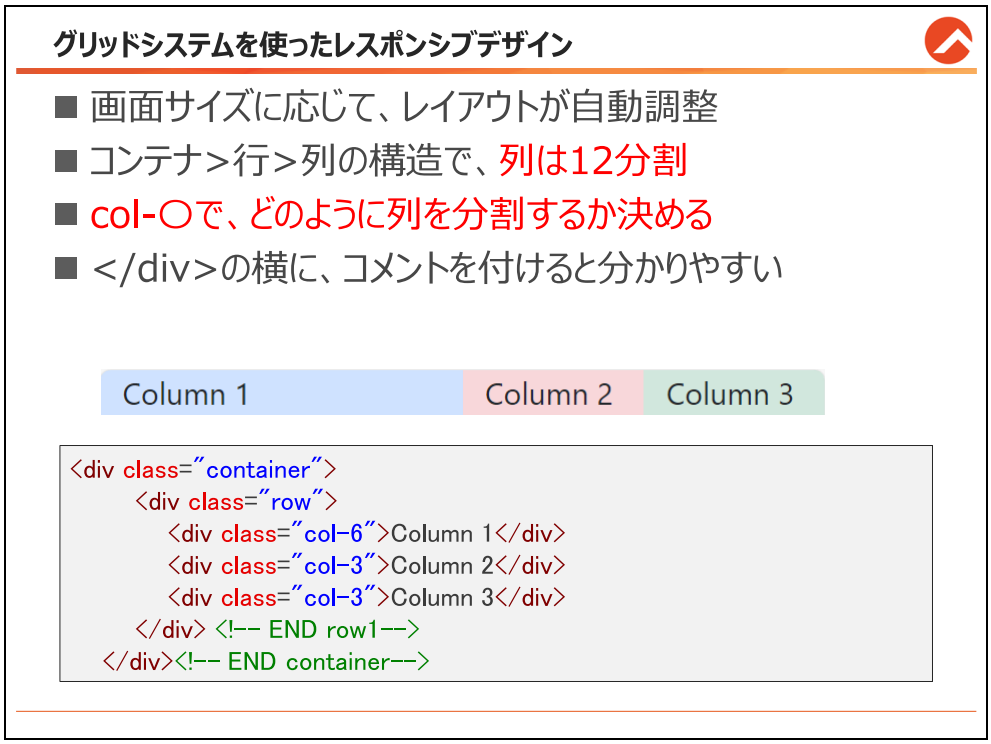
```
            <tr>
                <td class = "table-active">強調したいデータ</td>
                <td>データ8</td>
                <td>長いデータ長いデータ長いデータ長いデータ長いデータ
</td>
            </tr>
        </tbody> <!-- END table body -->
    </table> <!-- END table -->
</div> <!-- END responsive table wrapper -->
</div> <!-- END container -->
```

■動作確認

http://localhost:8080/7-4

class 属性を一つ消す度に、どう変わっていくのかを確認すると、よりわかりやすいです。

7.5. グリッドシステムを使ったレスポンシブデザイン



グリッドシステムの基本概念

Bootstrap のグリッドシステムを使うと、画面サイズに応じて自動的にレイアウトが調整される Web ページを簡単に作成できます。従来のCSS レイアウトで苦労していた、**ブロック要素の縦積みや横方向への配置が非常に簡単になります。**

Bootstrap の**グリッドシステムは、12 列**のシステムを採用しています。1 行（row）が最大で12 の列（column）に分割できるということです。このシステムを使って、柔軟でレスポンシブなレイアウトを作成できます。

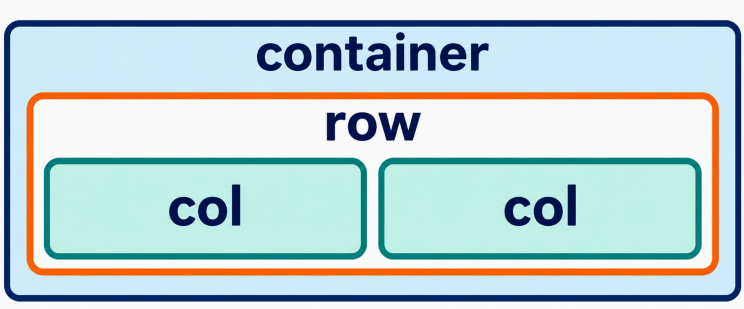
グリッドシステムでは、以下の3 つの基本要素を使います。

① コンテナ（Container）：**ページの中央に配置**され、内容を適切に配置するための基本的なレイアウト要素です。全体の幅を制御します。

② 行（Row）:カラム（列）を含む水平のレイアウト要素です。コンテナ要素の中に記述します。

③ カラム（Column）:列のことを「カラム」と言います。カラムは、コンテンツを配置するための垂直のレイアウト要素です。最大12 カラムまで使用できます。

図7. 5. 1



containerの中にrowを置き、rowの中にcolを並べます。

これらの要素は入れ子構造で、HTML 要素のclass 属性を使って指定します。

入れ子構造は、<table><tr><td>の関係と似ています。

基本的なグリッドレイアウト

① コンテナ（Container）

** **は、ページの中央にコンテンツを配置し、左右に適切な余白を追加するための基本的なレイアウト要素です。container を使用すると、幅が固定されており、**デバイスの画面サイズに応じて最大幅が変わります。コンテンツが常に中央に配置され**ますので、読みやすさが向上します。

一方、** **は、画面全体に広がるコンテナを作成します。余白なしで、親要素の**幅いっぱい**にコンテンツを広げたい場合に使用します。例えば、全幅のヘッダーやフッターを作成する場合に便利です。

② 行（Row）

** **は、水平に配置される行です。row の中にカラム（列）が入ります。row を使用することで、カラムが適切に並び、縦方向のスペースを均等に分けることができます。

③ カラム（Column）

** **のように、カラム（列）は、コンテンツを配置するための垂直のレイアウト要素です。カラムは、グリッドシステムの基礎となり、1 行を最大12 のカラムに分割することができます。以下に具体的な例を示します。

HTML …/resources/templates/grid_sample.html

```
<div class="container">
    <div class="row">
        <div class="col-6 bg-primary-subtle">Column 1</div>
        <div class="col-3 bg-danger-subtle">Column 2</div>
        <div class="col-3 bg-success-subtle">Column 3</div>
    </div> <!-- END row1-->
</div><!-- END container-->
```

コードの読み方

1行目

```
<div class="container">
```

今回のサンプルでは、外側を container でコンテナを作っています。画面の中央にコンテンツが配置されます。もし、container-fluid にすると、横幅いっぱいに変わります。

コードの読み方

2行目

```
<div class="row">
```

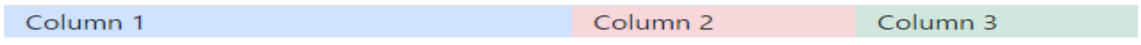
コンテナの中に、row をclass 属性として持つdiv 要素を入れます。後述のカラムを配置するためにも、必ずrow が必要です。まるでtable 要素に、<tr>がないと、<td>も書けないのと同じです。

3～5行目

```
<div class="col-6 bg-primary-subtle">Column 1</div>
<div class="col-3 bg-danger-subtle">Column 2</div>
<div class="col-3 bg-success-subtle">Column 3</div>
```

1 つのrow の中に、複数のカラムを配置できます。このサンプル通りだと、以下のように表示されます。

図7. 5. 2



** : このカラムは、12 分の6、つまり全幅の50%を占めます。**

** : このカラムは、12 分の3、つまり全幅の25%を占めます。**

Bootstrap のグリッドは12 列構成で、合計12 未満なら空白、12 超なら折り返し、均等配分には番号なしのcol を使います。

今回のサンプルの場合は、6 + 3 + 3 = 12 でした。

背景色はカラムの視覚的な区別を付けるために使用しています。

コードの読み方

6～7行目

```
    </div> <!-- END row1-->
</div><!-- END container-->
```

グリッドシステムでは、 要素を使うことが多いので、閉じタグだけでは、どの要素の閉じタグなのか、区別がつきません。そのため、**どの要素の閉じタグなのかを明確にするため、コメントを記載することが多いです。**

特に、row が複数ある場合は「END row1」や、「END row2」とかでコメントを残すことで実際の範囲も読みやすくなるので、重要です。

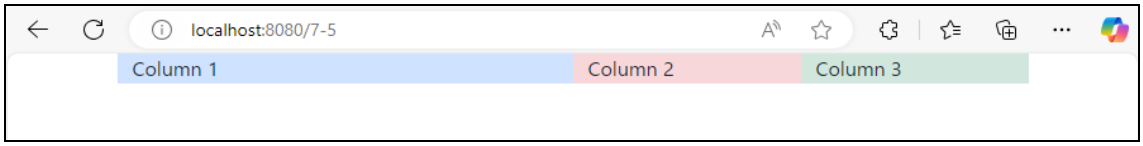
【参考】知っておくと便利になるカラムのコツ

- 12 未満なら、残りのスペースは自動的に分配されます。
- col-auto と設定すると、コンテンツの長さによって、横幅が自動的に計算されます。
- カラムを画面の中央寄せしたい時は、row クラスに加えて、「justify-content-center」クラスも適用します。
- g-1 (g-5 まである) をrow クラスに加えると、カラム同士の間隔を調整できます。微調整することが多いので、覚えておいてください。

■動作確認

http://localhost:8080/7-5

図7. 5. 3

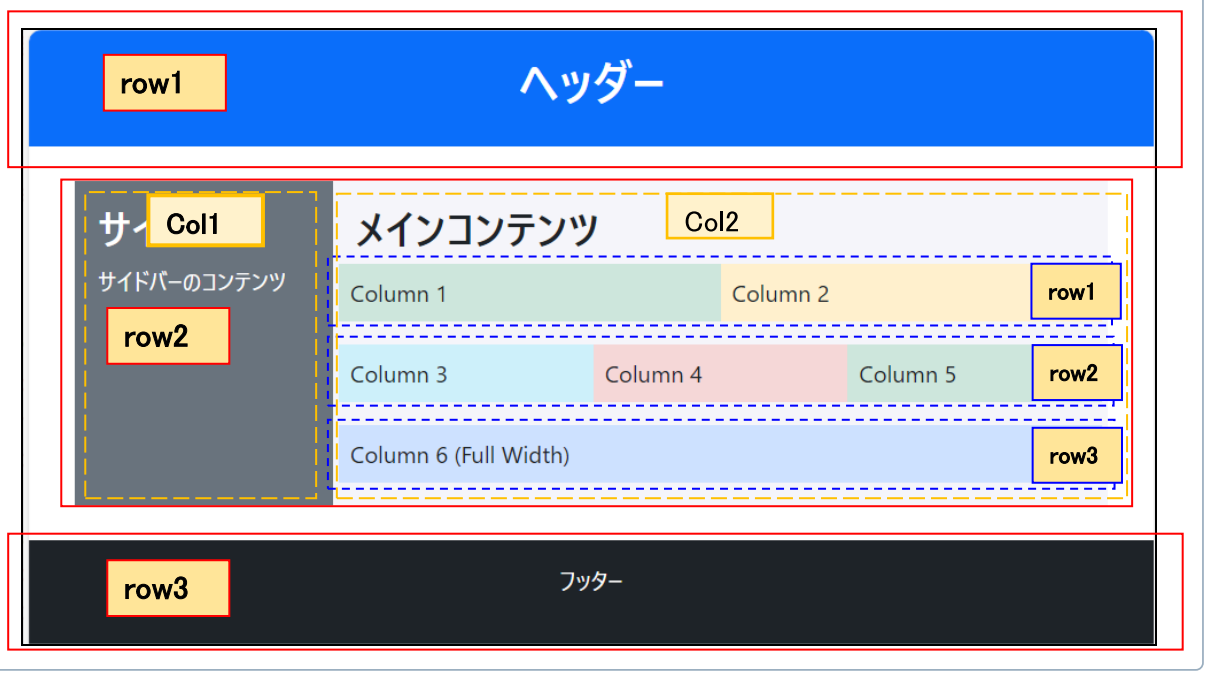


※画面の横幅を調整しても、同じ割合で要素のサイズが調整されることを確認してください。

※より詳しい構文は、https://getbootstrap.jp/docs/5.3/layout/grid/ を参照。

【参考】グリッドシステムの応用

図7. 5. 4



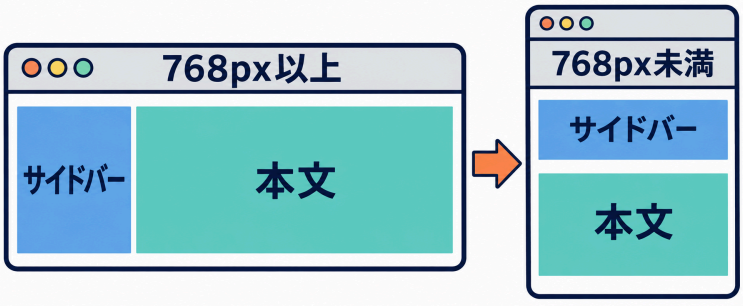
今まで学習した内容を応用すると、上のような画面を作成できます。

ポイントは、3 つです。

1. ヘッダーとフッターは、全幅に広がるようにするために「container-fluid」を使用し、サイドバーとメインコンテンツは中央に配置するために「container」を使用しています。
2. 各要素にはマージンとパディングを指定して、内部と外部に余白を入れています。
3. サイドバーとメインコンテンツは列(Column)を2 つで分けて1 つの行（row）に配置し、メインコンテンツの中にはさらに3 つの行（row）を入れ子にして配置しています。
4. col-md-3 とcol-md-9 のmd の意味について説明します。

横幅が長いPC 画面の場合は3:9 の割合が維持されますが、**タブレット端末以下の端末になった際は3:9 のレイアウトではなく、より最適なレイアウトに調整される**という意味です。sm にすると、スマホ以下の端末で調整されます。

図7. 5. 5



col-md-3／col-md-9では、幅768px以上は横並び、それ未満はサイドバー・本文の順に縦に並びます。

HTML …/resources/templates/grid_sample2.html

```
<!-- ヘッダー -->
<div class="container-fluid bg-primary text-white text-center py-3">
    <h1>ヘッダー</h1>
</div> <!-- END container1-->

<!-- メインコンテンツとサイドバー -->
<div class="container my-4">
    <div class="row">
        <div class="col-md-3 bg-secondary text-white p-3 mb-3 mb-md-0">
            <h2>サイドバー</h2>
            <p>サイドバーのコンテンツ</p>
        </div> <!-- END sidebar col-->
        <div class="col-md-9 bg-light p-3">
            <h2>メインコンテンツ</h2>
            <div class="row">
                <div class="col-6 p-2 bg-success-subtle">Column 1</div>
                <div class="col-6 p-2 bg-warning-subtle">Column 2</div>
            </div> <!-- END main col row1-->
            <div class="row mt-3">
                <div class="col-4 p-2 bg-info-subtle">Column 3</div>
                <div class="col-4 p-2 bg-danger-subtle">Column 4</div>
                <div class="col-4 p-2 bg-success-subtle">Column 5</div>
            </div> <!-- END main col row2-->
            <div class="row mt-3">
                <div class="col-12 p-2 bg-primary-subtle">Column 6 (Full Width)</div>
            </div> <!-- END main col row3-->
        </div> <!-- END main col-->
    </div> <!-- END row1-->
```

HTML …/resources/templates/grid_sample2.html 続き

```
29  </div> <!-- END container2-->
30
31  <!-- フッター -->
32  <div class="container-fluid bg-dark text-white text-center py-3">
33      <p>フッター</p>
34  </div> <!-- END container3-->
```

■動作確認

http://localhost:8080/7-5-2

図7.5.6



※画面の横幅を調整しても、同じ割合で要素のサイズが調整されることを確認してください。

※より詳しい構文は、https://getbootstrap.jp/docs/5.3/layout/columns/ を参照。

7.6. フォーム

フォーム

- 検索フォームと、登録フォームの実装方法が異なる
- 検索フォームの場合、グリッドシステムを利用してフォームとボタンを横並びにできる
- label要素は、入力フォームの説明を表示するための要素で、フォームと並べることが推奨される

種類	付与するclass名
ラベル	form-label form-check-label （radio/checkboxの項目ラベルに付与）
テキスト・パスワード	form-control
ラジオボタン・チェックボックス	form-check（グループに付与） form-check-inline（横並び用） form-check-input
セレクトメニュー	form-control

Bootstrap は、フォームのスタイルを簡単に整えるための豊富なクラスを提供しています。このクラスを使うことで、デザインが統一され、モダンなWeb ページを手軽に作成できます。

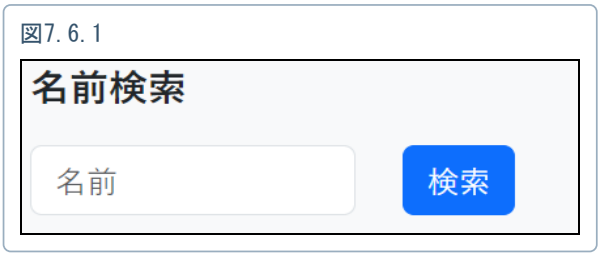
フォームの種類について

フォームには、検索バーのようなシンプルなフォームと、登録ページのように複数の入力フィールドを連続して配置する垂直フォームの2 種類のスタイルがあります。

検索フォーム

検索フォームは、ユーザが少量の情報を入力できるように設計されています。このフォームは、ページの一部として目立たせる必要があるため、シンプルかつ機能的なデザインが求められます。

図7. 6. 1



HTML …/resources/templates/form_sample.html

```
<body class="bg-light p-4">
<div class="container"> <!-- START container -->

    <div class="row"> <!-- START row1 -->
        <div class="col-md-3"> <!-- START sidebar -->
            <div class="fs-5 fw-bold mb-3">名前検索</div>

            <form class="row">
                <div class="col-8">
                    <input type="text" class="form-control"
                        placeholder="名前">
                </div>
                <div class="col-4">
                    <button type="submit"
                        class="btn btn-primary">検索</button>
                </div>
            </form>

        </div> <!-- END sidebar -->
    </div> <!-- END row1 -->
</div> <!-- END container -->
```

検索フォームは、前の節で学習した、グリッドシステムを利用します。

コードの読み方

5行目

```
<div class="col-md-3"> <!-- START sidebar -->
```

フォームとは直接関係がないコードですが、横幅の3 割のみ占めるように、限定をさせることで、左サイドバーのようなレイアウトを作ることができます。検索バーは、サイドバーに配置させることが多いので、予めカラムを生成しました。

8行目

```
<form class="row">
```

〈div〉ではなく、〈form〉要素をrow として生成すると、フォームを別途の行として扱えることができます。

9行目・13行目

```
<div class="col-8">
<div class="col-4">
```

入力フォームに8、ボタンに4 の割合でカラムを生成しています。

10行目

```
<input type="text" class="form-control"
```

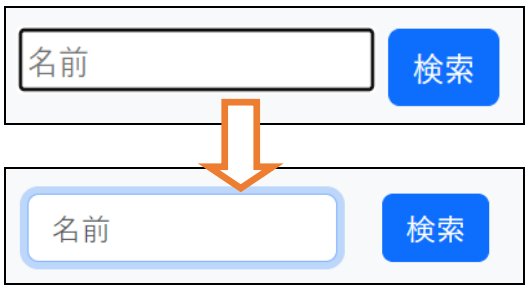
コードの読み方

10行目

```
<input type="text" class="form-control"
```

Bootstrap のフォーム関連クラスの中で、一番大事なクラスです。テキストフォームにこのクラスを付与することで、**Bootstrap 式のテキストフォームのデザインを適用**することができます。

図7.6.2



form-control 無しでフォームをクリック

form-control 付与後、フォームをクリック

他にも、form-control を付与することで、できることが色々ありますので、公式サイトを参照してください。

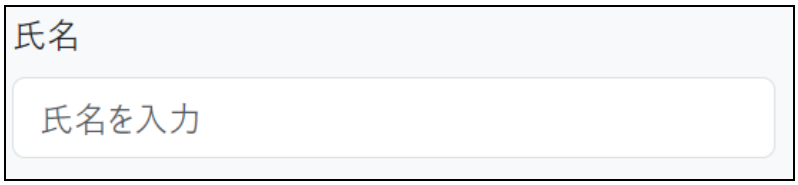
https://getbootstrap.jp/docs/5.3/forms/form-control/

登録ページのフォーム

登録ページでは、複数の入力フィールドを連続して配置する垂直フォームを使用します。以下に、よく使われるパターンの入力フィールドを1 つずつ紹介し、構文の説明とサンプルコードをご説明します。

1. テキストフォーム（input type = text、password）

図7. 6. 3



HTML

```
<label for="inputName" class="form-label">氏名</label>
<input type="text" class="form-control" id="inputName"
  placeholder="氏名を入力">
```

フォームには、**「ラベル」**という要素があります。ラベルとは、入力フォームの説明を表示するための要素です。ラベルを使わず、普通にテキストを配置してもいいですが、通常は、ラベルを使って説明する実装が推奨されます。

for 属性を使って、**関連付けたい入力フィールドのid を指定**すると、**ラベルをクリックした時、関連する入力フィールドが自動的にフォーカス**されます。

- **form-label クラス:**label 要素に適用されるクラスで、ラベルを適切にスタイルし、フォーム全体の見た目を整えます。
- **form-control クラス:**入力フィールドに適用されるクラスで、テキスト入力フィールドが Bootstrap のスタイルで適用されます。

2. ラジオボタン、チェックボックス（input type = radio、checkbox）

図7. 6. 4



HTML

```
<label class="form-label">性別</label><br>
<div class="form-check form-check-inline">
      <input type="radio" class="form-check-input" id="genderMale"
      name="gender" value="male">
      <label for="genderMale" class="form-check-label">男性
</label>
</div>

<div class="form-check form-check-inline">
      <input type="radio" class="form-check-input"
id="genderFemale"
      name="gender" value="female">
      <label for="genderFemale" class="form-check-label">女性
</label>
</div>
```

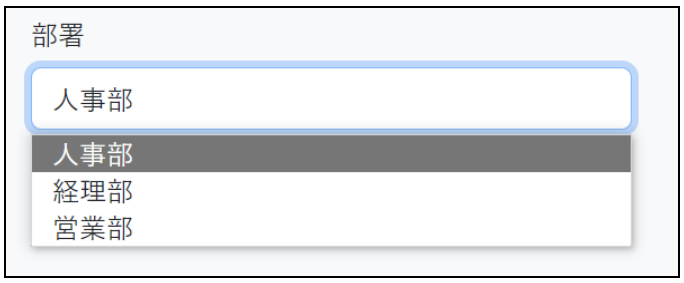
ラジオボタンをBootstrap実装する際は、2つの種類のラベルが必要です。「性別」というラベルと、「男性・女性」という個別のラベルを実装する必要があります。

ラジオボタンと、チェックボックスは必要に応じて、以下のクラスを適用します。

- form-check **クラス**:チェックボックスやラジオボタンのグループに適用されるクラスで、適切な間隔とスタイルを設定します。
- **form-check-inline クラス:**form-check クラスに加えてform-check-inline クラスを使用することで、ラジオボタンをインラインブロック要素にし、**横並びに配置**します。削除したら、改行されます。
- form-check-input **クラス**:チェックボックスやラジオボタンに共通のデザインを適用できます。
- form-check-label **クラス**:チェックボックスやラジオボタンのラベルに適用します。for 属性と連携するので、**レベルをクリックするだけでも、ラジオボタンがチェックされる**便利な UI を作ります。

3. セレクトメニュー(select-option) :

図7.6.5



HTML

```
<label for="department" class="form-label">部署</label>
<select class="form-select" id="department">
        <option value="hr">人事部</option>
        <option value="finance">経理部</option>
        <option value="sales">営業部</option>
</select>
```

セレクトメニューは、<select>要素に、「form-select」クラスを適用するくらいです。

登録フォームのサンプルコード

HTML …/resources/templates/form_sample2.html

```
<body class="bg-light p-4">

<div class="container"> <!-- START container -->
    <form class="row justify-content-center"> <!-- START form row -->
        <div class="col-md-4"> <!-- START form column -->
            <h2 class="mb-4">登録フォーム</h2>

            <div class="mb-3"> <!--テキストフォーム-->
                <label for="inputName" class="form-label">氏名</label>
                <input type="text" class="form-control" id="inputName"
                    placeholder="氏名を入力">
            </div>
            <div class="mb-3"> <!--パスワードフォーム-->
                <label for="inputPassword"
                    class="form-label">パスワード</label>
                <input type="password" class="form-control"
                    id="inputPassword" placeholder="パスワードを入力">
```

HTML

```
            </div>
            <div class="mb-3"> <!--ラジオボタン-->
                <label class="form-label">性別</label><br>
                <div class="form-check form-check-inline">
                    <input type="radio" class="form-check-input"
                        id="genderMale" name="gender"
value="male">
                    <label for="genderMale"
                        class="form-check-label">男性</label>
                </div>
                <div class="form-check form-check-inline">
                    <input type="radio" class="form-check-input"
                        id="genderFemale" name="gender"
value="female">
                    <label for="genderFemale"
                        class="form-check-label">女性</label>
                </div>
            </div>
            <div class="mb-3"> <!--チェックボックス-->
                <div class="form-check">
                    <input type="checkbox" class="form-check-
input"
                        id="subscribeNewsletter">
                    <label for="subscribeNewsletter"
                       class="form-check-label">ニュースレターを購読
する</label>
                </div>
            </div>
            <div class="mb-3"> <!--セレクトメニュー-->
                <label for="department" class="form-label">部署
</label>
                <select class="form-select" id="department">
                    <option value="hr">人事部</option>
                    <option value="finance">経理部</option>
                    <option value="sales">営業部</option>
                </select>
```

HTML 続き

```
            </div>
            <button type="submit" class="btn btn-primary">登録
</button>
        </div> <!-- END form column -->
    </form> <!-- END form row -->
</div> <!-- END container -->
```

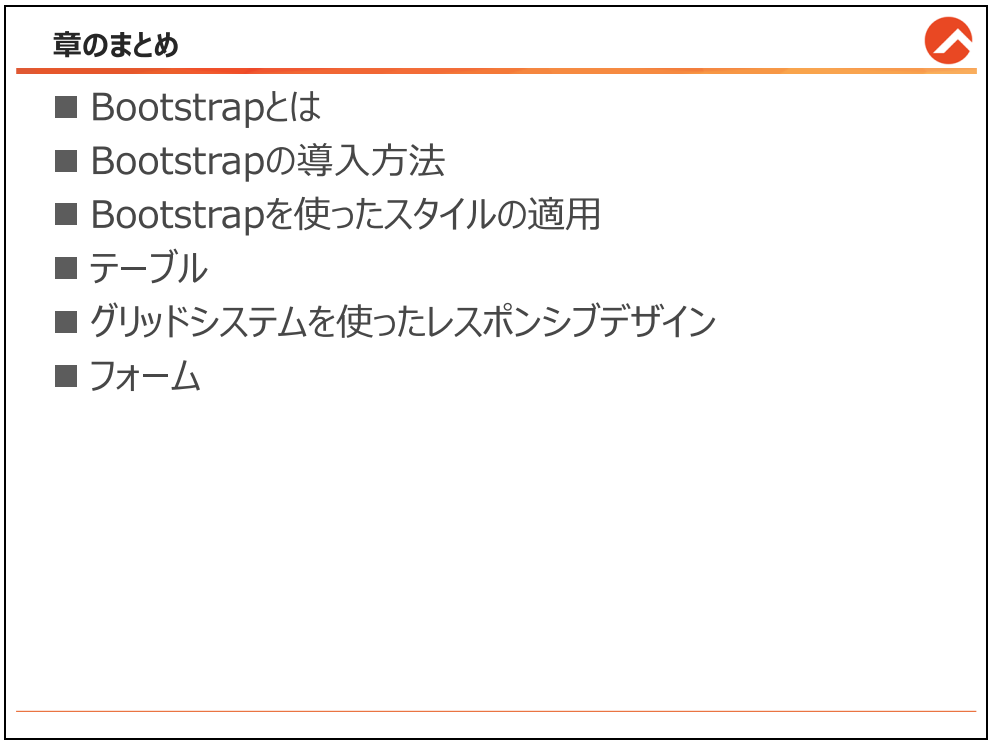
■動作確認

図7. 6. 6



- 画面のサイズを変えても、配置が維持されることを確認してください。
- 各フォームが統一されたデザインになっていることを確認してください。
- ラジオボタンと、セレクトメニューが正常動作することを確認してください。

章のまとめ



この章では、Bootstrap についてご説明しました。本書で説明した内容は、全てを網羅しているわけではありませんので、今回学習した知識を基に、必要に応じて公式サイトなどを調べながら実装する習慣をつけてください。